SCHOOL OF COMPUTER SCIENCE AND ENGINEERING (SCOPE)

BACSE202 Lab Component Project – Assessment 8

HOTEL MANAGEMENT SYSTEM

A relational database design and implementation using PostgreSQL

THIS REPORT PRESENTS

1 Case Study
2 Abstract
3 System Design
4 ER Diagram
5 Workflow

PROJECT TITLE
HOTEL MANAGEMENT SYSTEM

SUBMITTED BY
SHRADDHA SIDHAN

REGISTRATION NUMBER
25BDS0110

COURSE / PROJECT
BACSE202 Lab Component Project – Assessment 8

ACADEMIC YEAR
2026–27

DATABASE PLATFORM
PostgreSQL 17 (SQL)

Contents

1 Case Study . . . 3
1.1 The Real-World Problem . . . 3
1.2 Limitations of Register-Based Systems and Spreadsheets . . . 3
1.3 The Proposed Hotel Management System . . . 4
1.4 Objectives . . . 4
1.5 Scope . . . 5
1.6 Expected Benefits . . . 5
2 Abstract . . . 6
3 System Design . . . 7
3.1 Design Approach . . . 7
3.2 System Architecture . . . 8
3.3 Entities of the Database . . . 9
3.4 Data Dictionary . . . 9
3.5 Keys and Constraints . . . 11
3.6 Normalisation up to Third Normal Form . . . 11
4 ER Diagram . . . 13
4.1 Overview and Notation . . . 13
4.2 Entity–Relationship Diagram . . . 14
4.3 Relationships and Cardinality . . . 15
5 Workflow . . . 16
5.1 Overview of the Stages . . . 16
5.2 Workflow Flowchart . . . 17
5.3 Explanation of Each Stage . . . 18
5.4 Status Changes During the Workflow . . . 19

List of Figures

Figure	Title	Section
Figure 1	Layered architecture of the Hotel Management System	3.2
Figure 2	Entity–Relationship diagram (Crow's Foot notation)	4.2
Figure 3	Workflow of the Hotel Management System	5.2

SECTION 1

Case Study

This case study examines how a typical mid-sized hotel manages its daily operations without an integrated database, identifies the limitations of that approach, and presents the Hotel Management System as a relational database solution to those limitations.

1.1 The Real-World Problem

A hotel is one of the most information-intensive service organisations. At any moment the front office must know which rooms are vacant, occupied or under maintenance; which guests are due to arrive or check out; which chargeable services each guest has used; how much each guest owes; and which employee is responsible for each task. These facts change continuously through the day and are closely interconnected: a booking links a guest to a room for a period of time, a payment settles a booking, and a service request links a booking to a service and to the employee who attended it.

Consider the kind of hotel modelled in this project. Its rooms are classified into categories such as Standard, Deluxe, Suite, Presidential and Family, each with its own tariff and occupancy capacity. Its staff work in departments such as Front Office, Housekeeping, Food and Beverage, Human Resources and Maintenance, within a reporting hierarchy headed by a General Manager. Guests use chargeable services such as room service, laundry, spa, airport pickup and the conference hall, and settle their bills by cash, card, UPI or net banking, often as an advance followed by a final payment.

In many small and medium hotels this information is still kept in a physical guest register at the reception, a separate room-status board, handwritten service slips and a billing book. Larger establishments may use spreadsheets, but these are usually stored on individual computers and are rarely synchronised between departments.

1.2 Limitations of Register-Based Systems and Spreadsheets

The manual arrangement described above leads to the following limitations:

Limitation	Consequence for the hotel
Data redundancy	The same guest's name, phone number and address are rewritten on the register, the service slip and the bill. A correction made in one place is not reflected in the others.
Data inconsistency	The room-status board and the register frequently disagree, so a room may be allotted to two guests or a vacant room may remain unsold.
No integrity enforcement	Nothing prevents a clerk from writing a check-out date earlier than the check-in date, entering a negative tariff or billing a room number that does not exist.
Poor concurrent access	Only one person can write in the register at a time, creating a bottleneck during peak check-in hours.
Difficult retrieval	A question such as "how much revenue did Deluxe rooms generate last month?" requires reading through weeks of handwritten entries.
Weak security	Anyone with physical access to the register can read or alter every guest's personal information.

Limitation	Consequence for the hotel
No analytical capability	Management has no reliable basis for pricing, staffing or offers to repeat guests.

Spreadsheets reduce clerical effort but not these underlying problems. Each sheet is an independent file: a booking sheet cannot verify that the guest or room it refers to actually exists, and nothing stops a cell from holding a misspelt status or an impossible date. Copies held by different departments drift apart, and every combined report requires manual merging of files.

1.3 The Proposed Hotel Management System

The proposed system replaces the register and the spreadsheets with a single, normalised relational database implemented in **PostgreSQL 17** and manipulated using **SQL**. The hotel's information is organised into nine related tables — GUEST, ROOM_TYPE, ROOM, BOOKING, PAYMENT, DEPARTMENT, EMPLOYEE, SERVICE and SERVICE_REQUEST — each of which records one kind of fact exactly once. Relationships between facts are represented by foreign keys, and business rules are declared as constraints in the schema, so that the database itself rejects invalid data. The table below shows how each limitation is addressed.

Problem	How the relational database solves it
Redundancy	Each fact is stored once in its own relation and referenced elsewhere by a foreign key; for example, a guest's phone number is held only in GUEST.
Inconsistency	All departments work on the same centralised data; the status of every room is held in one place, ROOM.ROOM_STATUS.
Integrity	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK and DEFAULT constraints make invalid states impossible rather than merely discouraged, e.g. CHECK (CHECKOUT_DATE > CHECKIN_DATE).
Concurrency	The PostgreSQL server lets several authorised users read and update data at the same time under transactional control.
Retrieval and reporting	SQL joins, aggregate functions, GROUP BY, HAVING and sub-queries answer management questions in a single statement.
Security	Data is held on a database server and accessed only by authorised database users, instead of lying open in a register.

1.4 Objectives

1. To design a relational schema in Third Normal Form for core hotel operations: guests, rooms, bookings, payments, staff and services.
2. To enforce data integrity at the database level through primary keys, foreign keys and domain constraints.
3. To support the complete guest lifecycle: registration, availability check, reservation, stay, service requests, billing, payment and check-out.
4. To maintain employee records by department, including a self-referencing manager hierarchy.
5. To derive management information such as revenue per room category, occupancy, highest-spending guests, department-wise salary expenditure and the most frequently requested services.

6. To demonstrate the DDL, DML and DQL capabilities of SQL on a working PostgreSQL database.

1.5 Scope

The project covers four functional areas of hotel operation:

- **Front Office Management** — registration of guests with identity proof and contact details, allotment of rooms by category and availability, and recording of check-in and check-out dates.
- **Room Management** — the room inventory, classification of rooms into categories with their tariffs and occupancy capacity, and tracking of the current status of each room.
- **Service Management** — chargeable services such as laundry, room service, spa, airport pickup and conference hall booking, together with the employee who attended each request.
- **Staff and Billing Management** — employee records under departments with a reporting hierarchy, bills against bookings, and payments made through different modes.

The project concentrates on the database layer. The schema, constraints, sample records and queries are implemented and executed in PostgreSQL through the psql command-line interface. A graphical user interface, an online booking portal and payment-gateway integration are outside the scope of this report, although the schema can serve as the foundation for such applications.

1.6 Expected Benefits

- **Single entry of data** — guest, room, service and staff details are recorded once and reused by every booking, request and payment.
- **Reliable room information** — one authoritative room status reduces double allotment and unsold vacant rooms.
- **Accurate billing** — room charges follow the category tariff and service charges follow the service catalogue, so bills are computed rather than written by hand.
- **Valid data by design** — impossible dates, negative amounts and unknown statuses are rejected by the database.
- **Faster front-desk operations** — any booking, guest history or outstanding balance can be retrieved instantly.
- **Decision support** — revenue, occupancy, staffing and service-usage reports are produced directly from operational data.
- **Easy maintenance** — a tariff revision for a whole category changes a single ROOM_TYPE row.

SECTION 2

Abstract

Hotels handle a continuous flow of interrelated information — guest identities, room inventory and status, reservations, payments, staff and chargeable services. When this information is kept in registers and disconnected spreadsheets it becomes redundant, inconsistent and difficult to analyse. This project presents the design and implementation of a **Hotel Management System** as a normalised relational database built with **PostgreSQL 17** and the **Structured Query Language (SQL)**.

The database comprises nine entities: GUEST, ROOM_TYPE, ROOM, BOOKING, PAYMENT, DEPARTMENT, EMPLOYEE, SERVICE and SERVICE_REQUEST. The schema was derived from an Entity–Relationship model and normalised to Third Normal Form so that every fact is stored only once. Referential integrity is enforced through primary and foreign keys, including a recursive manager relationship within EMPLOYEE and the associative entity SERVICE_REQUEST, which resolves the many-to-many relationship between bookings and services. NOT NULL, UNIQUE, CHECK and DEFAULT constraints prevent invalid data such as non-positive tariffs, unrecognised room or booking statuses, and check-out dates that do not follow check-in dates.

The implementation demonstrates table creation with constraints, insertion of sample records, update and deletion operations, and retrieval using single-row functions, aggregate functions, GROUP BY, HAVING, sub-queries and multi-table joins. These operations support the full guest lifecycle, from registration and booking to service requests, billing, payment and check-out, and produce management information such as revenue by room category, occupancy status, department-wise salary expenditure and service usage. The expected outcome is a consistent, non-redundant and easily queryable data store that reduces manual error, speeds up front-desk work and gives management a reliable basis for decisions.

Keywords: Hotel Management System; relational database; PostgreSQL; SQL; Entity–Relationship model; normalisation; Third Normal Form; referential integrity.

SECTION 3

System Design

System design translates the requirements identified in the case study into a structure that can be implemented. This section first describes the design approach and the overall architecture of the system, and then the logical design of the database: its entities, attributes, keys, constraints and normalisation.

3.1 Design Approach

The database was developed through the standard database design life cycle shown below. Requirements gathered from hotel operations were modelled as an Entity–Relationship diagram; the entities and relationships were mapped to relations with primary and foreign keys; the relations were normalised to Third Normal Form; and the schema was implemented and tested in PostgreSQL.



3.2 System Architecture

The system follows a layered client–server architecture (Figure 1). Users never write to the data files directly; every request passes as an SQL statement through the PostgreSQL server, which validates it against the declared constraints before any data is changed.

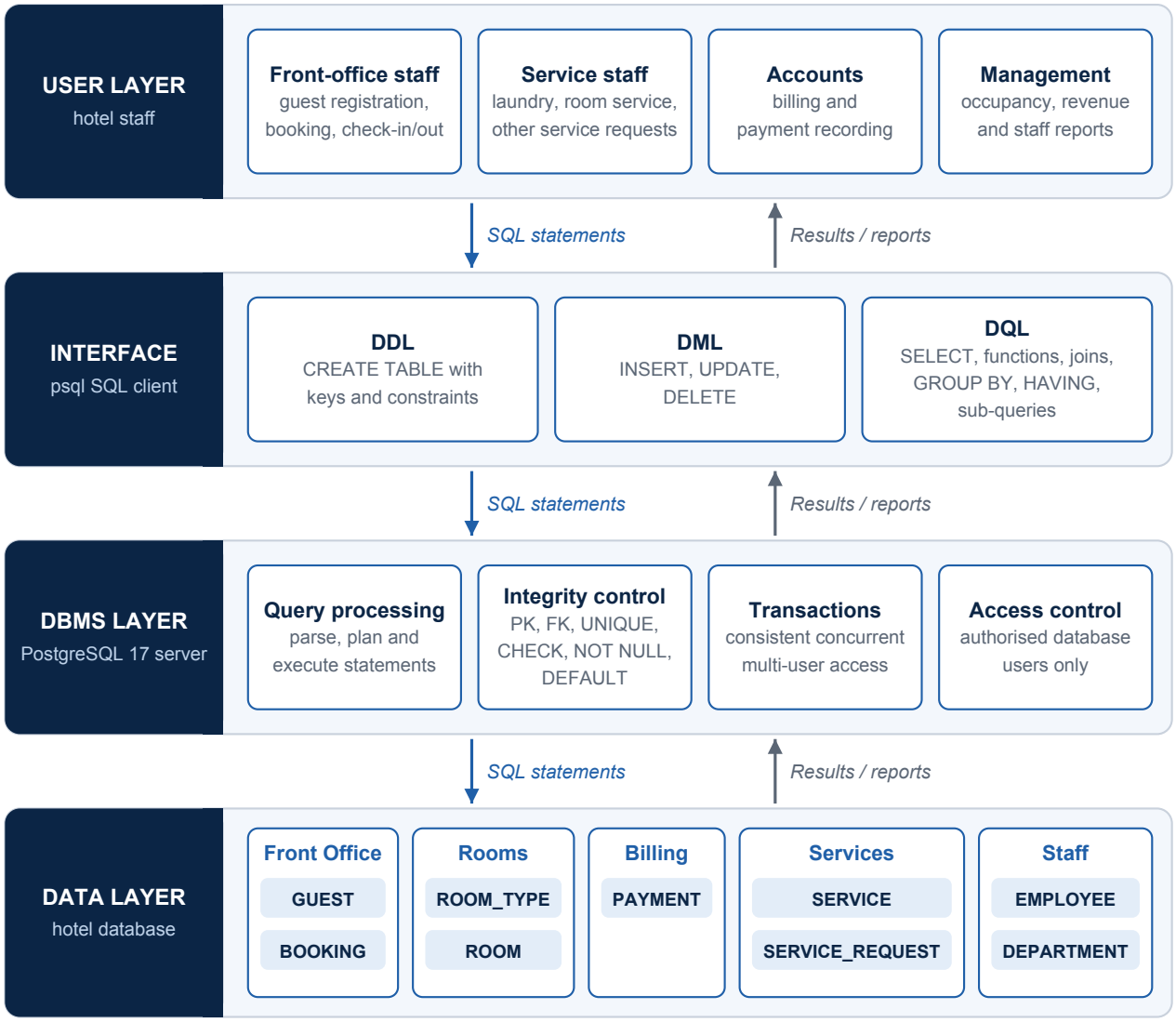


Figure 1: Layered architecture of the Hotel Management System

Layer	Role in the system
User layer	Hotel staff in different roles: front-office staff register guests and manage bookings, service staff record service requests, accounts staff record payments, and management retrieves reports.
Interface layer	The PostgreSQL psql command-line interface, through which SQL statements are submitted: DDL to define the schema, DML to insert, update and delete records, and DQL to retrieve information.
DBMS layer	The PostgreSQL 17 server parses, plans and executes each statement; checks every insert and update against the key and domain constraints; manages transactions so that concurrent users see consistent data; and admits only authorised database users.
Data layer	The hotel database of nine relations, grouped by functional area: front office, rooms, billing, services and staff.

Choice of technology. PostgreSQL 17, as used in the project report, was retained. It is an open-source relational DBMS with comprehensive support for SQL, primary and foreign keys, CHECK constraints, aggregate functions, sub-queries and joins, all of which the Hotel Management System requires. It provides reliable data integrity and is freely available for academic use.

3.3 Entities of the Database

The database consists of nine entities. Each is implemented as a table whose primary key is a single integer identifier.

Entity	Purpose	Primary key	Foreign keys (references)
GUEST	Personal and contact details of each guest, including identity proof	GUEST_ID	—
ROOM_TYPE	Room categories with base tariff and maximum occupancy	TYPE_ID	—
ROOM	Physical rooms with room number, floor and current status	ROOM_ID	TYPE_ID → ROOM_TYPE
BOOKING	Reservation of a room by a guest for a stay period, with amount and status	BOOKING_ID	GUEST_ID → GUEST ROOM_ID → ROOM
PAYMENT	Payment made against a booking, with amount, date and mode	PAYMENT_ID	BOOKING_ID → BOOKING
DEPARTMENT	Hotel departments and their locations	DEPT_ID	—
EMPLOYEE	Staff details, department and reporting manager	EMP_ID	DEPT_ID → DEPARTMENT MANAGER_ID → EMPLOYEE
SERVICE	Catalogue of chargeable services and their prices	SERVICE_ID	—
SERVICE_REQUEST	A service consumed during a booking, with quantity, date and attending employee (associative entity)	REQUEST_ID	BOOKING_ID → BOOKING SERVICE_ID → SERVICE EMP_ID → EMPLOYEE

3.4 Data Dictionary

The attributes, data types and column-level constraints of each table are listed below, exactly as defined in the project schema.

Table	Attribute	Data type	Constraints
GUEST	GUEST_ID	INT	PRIMARY KEY
	GUEST_NAME	VARCHAR(50)	NOT NULL
	PHONE	VARCHAR(15)	UNIQUE, NOT NULL
	EMAIL	VARCHAR(100)	UNIQUE
	ADDRESS	VARCHAR(100)	—
	CITY	VARCHAR(50)	—
	ID_PROOF_NO	VARCHAR(30)	UNIQUE, NOT NULL

Table	Attribute	Data type	Constraints
ROOM_TYPE	TYPE_ID	INT	PRIMARY KEY
	TYPE_NAME	VARCHAR(30)	UNIQUE, NOT NULL
	BASE_PRICE	NUMERIC(10,2)	NOT NULL, CHECK (BASE_PRICE > 0)
	CAPACITY	INT	NOT NULL, CHECK (CAPACITY > 0)
ROOM	ROOM_ID	INT	PRIMARY KEY
	ROOM_NO	INT	UNIQUE, NOT NULL
	FLOOR_NO	INT	NOT NULL, CHECK (FLOOR_NO > 0)
	ROOM_STATUS	VARCHAR(20)	DEFAULT 'Available'; CHECK IN ('Available', 'Occupied', 'Maintenance')
	TYPE_ID	INT	NOT NULL, FOREIGN KEY → ROOM_TYPE
DEPARTMENT	DEPT_ID	INT	PRIMARY KEY
	DEPT_NAME	VARCHAR(50)	UNIQUE, NOT NULL
	LOCATION	VARCHAR(50)	—
EMPLOYEE	EMP_ID	INT	PRIMARY KEY
	EMP_NAME	VARCHAR(50)	NOT NULL
	DESIGNATION	VARCHAR(50)	—
	SALARY	NUMERIC(10,2)	CHECK (SALARY > 0)
	HIRE_DATE	DATE	NOT NULL
	DEPT_ID	INT	NOT NULL, FOREIGN KEY → DEPARTMENT
	MANAGER_ID	INT	FOREIGN KEY → EMPLOYEE (nullable)
BOOKING	BOOKING_ID	INT	PRIMARY KEY
	BOOKING_DATE	DATE	NOT NULL
	CHECKIN_DATE	DATE	NOT NULL
	CHECKOUT_DATE	DATE	NOT NULL; table CHECK (CHECKOUT_DATE > CHECKIN_DATE)
	TOTAL_AMOUNT	NUMERIC(10,2)	CHECK (TOTAL_AMOUNT >= 0)
	BOOKING_STATUS	VARCHAR(20)	DEFAULT 'Confirmed'; CHECK IN ('Confirmed', 'Checked-In', 'Checked-Out', 'Cancelled')
	GUEST_ID	INT	NOT NULL, FOREIGN KEY → GUEST
	ROOM_ID	INT	NOT NULL, FOREIGN KEY → ROOM
PAYMENT	PAYMENT_ID	INT	PRIMARY KEY
	AMOUNT	NUMERIC(10,2)	NOT NULL, CHECK (AMOUNT > 0)
	PAY_DATE	DATE	NOT NULL
	PAY_MODE	VARCHAR(20)	CHECK IN ('Cash', 'Card', 'UPI', 'Net Banking')
	BOOKING_ID	INT	NOT NULL, FOREIGN KEY → BOOKING
SERVICE	SERVICE_ID	INT	PRIMARY KEY
	SERVICE_NAME	VARCHAR(50)	UNIQUE, NOT NULL
	SERVICE_PRICE	NUMERIC(10,2)	NOT NULL, CHECK (SERVICE_PRICE > 0)

Table	Attribute	Data type	Constraints
SERVICE_REQUEST	REQUEST_ID	INT	PRIMARY KEY
	QUANTITY	INT	NOT NULL, CHECK (QUANTITY > 0)
	REQUEST_DATE	DATE	NOT NULL
	BOOKING_ID	INT	NOT NULL, FOREIGN KEY → BOOKING
	SERVICE_ID	INT	NOT NULL, FOREIGN KEY → SERVICE
	EMP_ID	INT	NOT NULL, FOREIGN KEY → EMPLOYEE

3.5 Keys and Constraints

Primary keys. Every table has a single-column integer primary key (GUEST_ID, TYPE_ID, ROOM_ID, BOOKING_ID, PAYMENT_ID, DEPT_ID, EMP_ID, SERVICE_ID and REQUEST_ID). A primary key is implicitly unique and NOT NULL, so every row can be identified unambiguously and referenced from other tables.

Foreign keys. Nine foreign keys implement the nine relationships of the ER model. Each foreign key is placed on the "many" side of a one-to-many relationship and must match an existing primary key, so a booking cannot refer to a non-existent guest or room, and a payment cannot refer to a non-existent booking. MANAGER_ID refers to EMP_ID in the same table, forming the recursive manager relationship; it is the only nullable foreign key, because the General Manager reports to no one within the hotel.

Domain constraints. The remaining business rules are declared in the schema, as summarised below.

Constraint	Purpose	Where it is applied
UNIQUE	Candidate (alternate) keys; no duplicates	GUEST.PHONE, GUEST.EMAIL, GUEST.ID_PROOF_NO, ROOM_TYPE.TYPE_NAME, ROOM.ROOM_NO, DEPARTMENT.DEPT_NAME, SERVICE.SERVICE_NAME
NOT NULL	Mandatory values	Names, dates, prices, quantities and every foreign key except MANAGER_ID
CHECK (range)	Positive or non-negative numbers	BASE_PRICE, CAPACITY, FLOOR_NO, SALARY, AMOUNT, SERVICE_PRICE, QUANTITY > 0; TOTAL_AMOUNT >= 0
CHECK (list)	Only recognised values	ROOM_STATUS, BOOKING_STATUS and PAY_MODE
CHECK (table)	Rule involving two columns	CHECKOUT_DATE > CHECKIN_DATE in BOOKING
DEFAULT	Initial value when none is given	ROOM_STATUS = 'Available'; BOOKING_STATUS = 'Confirmed'

3.6 Normalisation up to Third Normal Form

First Normal Form (1NF). Every attribute holds a single atomic value and every table has a primary key, so there are no repeating groups. A guest who makes three bookings produces three rows in BOOKING rather than three booking columns in GUEST, and each service consumed during a stay is a separate row in SERVICE_REQUEST rather than a list stored inside BOOKING.

Second Normal Form (2NF). A partial dependency can arise only when a primary key is composite. All nine tables have single-attribute primary keys, so every non-key attribute depends on the whole

key and the schema is in 2NF. SERVICE_REQUEST, for instance, uses REQUEST_ID, and QUANTITY and REQUEST_DATE describe that request as a whole.

Third Normal Form (3NF). No non-key attribute depends on another non-key attribute. The table below shows the transitive dependencies that would have existed in an unnormalised design and how they were removed by placing each fact in its own relation.

Dependency avoided	Why it would be transitive	Resolution
ROOM_ID → TYPE_ID → BASE_PRICE, CAPACITY	Tariff and capacity depend on the room category, not on the individual room	Stored once in ROOM_TYPE; ROOM keeps only TYPE_ID
EMP_ID → DEPT_ID → DEPT_NAME, LOCATION	Department details depend on the department, not on the employee	Stored in DEPARTMENT; EMPLOYEE keeps only DEPT_ID
REQUEST_ID → SERVICE_ID → SERVICE_NAME, SERVICE_PRICE	Price depends on the service, not on the individual request	Stored in SERVICE; SERVICE_REQUEST keeps only SERVICE_ID
BOOKING_ID → GUEST_ID → GUEST_NAME, PHONE, …	Contact details belong to the guest, not to the booking	Stored in GUEST; BOOKING keeps only GUEST_ID

As a result, a tariff revision for an entire category requires the update of a single ROOM_TYPE row, and insertion, update and deletion anomalies are avoided. BOOKING.TOTAL_AMOUNT records the amount agreed for the stay when the booking is made, so later tariff revisions do not alter the history of past bookings.

SECTION 4

ER Diagram

4.1 Overview and Notation

The Entity–Relationship model of the Hotel Management System consists of nine entities connected by nine relationships, covering every functional area of the hotel: guest handling, room inventory, reservation, billing, staff administration and services. It satisfies the requirement of a minimum of seven entities.

The diagram on the next page uses **Crow's Foot notation**. A single bar on a connector marks the "one" side of a relationship, and a three-pronged crow's foot marks the "many" side; the numerals 1 and M are printed beside them for clarity. Every crow's foot ends at the entity that carries the corresponding foreign key. Within each entity box, primary keys are marked **PK** (and underlined), foreign keys **FK** (in italics) and unique keys **UK**.

SERVICE_REQUEST is drawn with a blue header because it is an **associative entity**. A booking may consume several services and a service may be consumed by several bookings, which is a many-to-many relationship that cannot be stored directly in a relational table. SERVICE_REQUEST converts it into two one-to-many relationships (BOOKING Raises SERVICE_REQUEST and SERVICE Requested as SERVICE_REQUEST) and also holds the attributes of the relationship itself: the quantity, the request date and the employee who attended the request.

EMPLOYEE participates in a **recursive relationship**, "Reports to": MANAGER_ID is a foreign key referring to EMP_ID in the same table, which models the supervisory hierarchy without a separate manager table.

4.2 Entity–Relationship Diagram

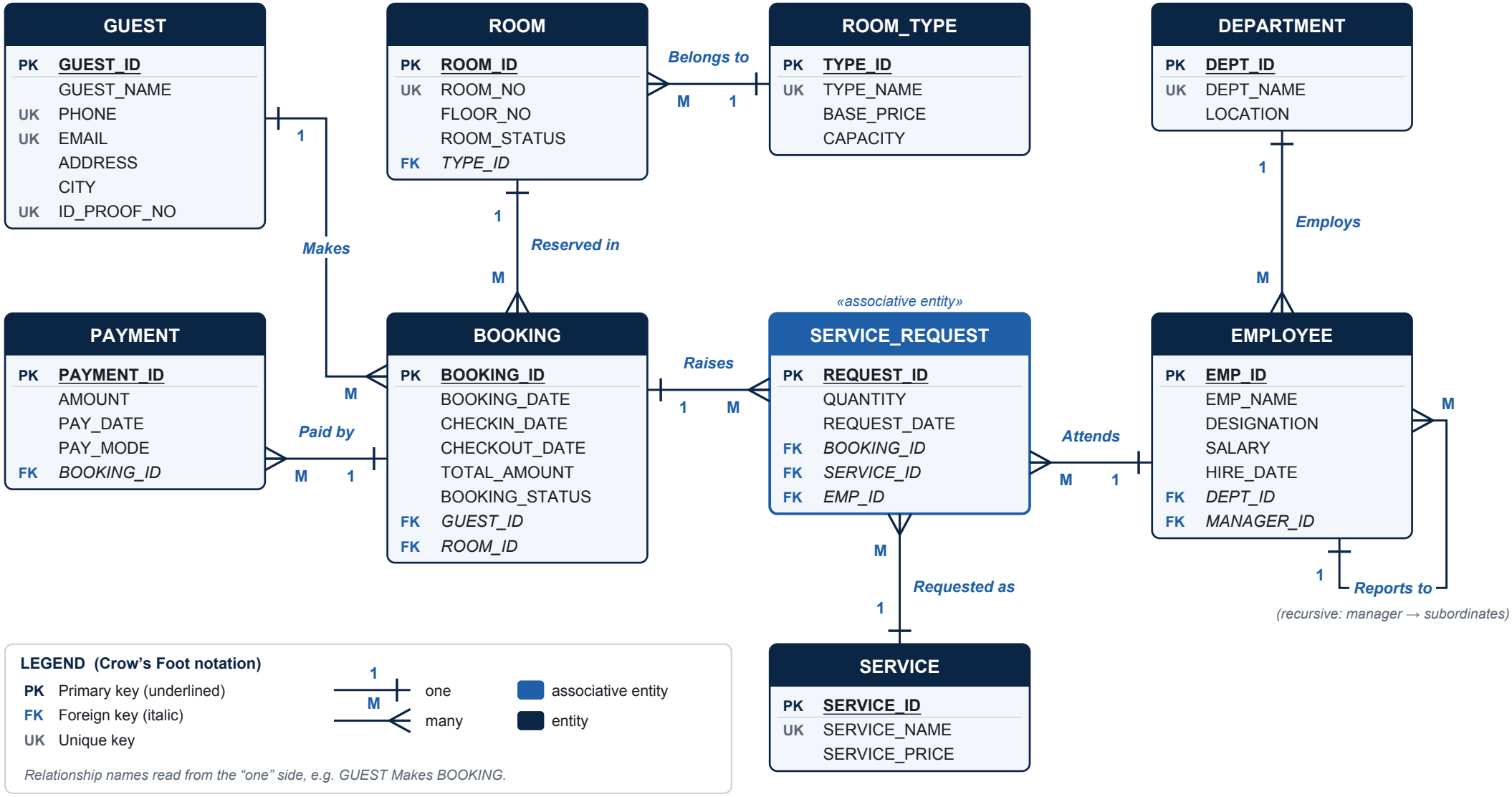


Figure 2: Entity–Relationship diagram of the Hotel Management System (Crow's Foot notation)

4.3 Relationships and Cardinality

The table below lists each relationship in Figure 2 with its cardinality and the foreign key that implements it. The foreign key is always held by the entity on the "many" side.

Relationship	One (parent)	Many (child)	Foreign key in child	Meaning
Belongs to	ROOM_TYPE	ROOM	TYPE_ID	One category is assigned to many physical rooms.
Makes	GUEST	BOOKING	GUEST_ID	One guest can make many reservations over time.
Reserved in	ROOM	BOOKING	ROOM_ID	One room is reserved many times across different dates.
Paid by	BOOKING	PAYMENT	BOOKING_ID	One booking may have an advance and a final payment.
Employs	DEPARTMENT	EMPLOYEE	DEPT_ID	One department employs many staff members.
Reports to (recursive)	EMPLOYEE (manager)	EMPLOYEE (subordinate)	MANAGER_ID	One manager supervises many employees.
Raises	BOOKING	SERVICE_REQUEST	BOOKING_ID	One booking can raise many service requests.
Requested as	SERVICE	SERVICE_REQUEST	SERVICE_ID	One service appears in many requests.
Attends	EMPLOYEE	SERVICE_REQUEST	EMP_ID	One employee attends many service requests.

All nine relationships are one-to-many, and the only many-to-many association in the problem domain, between BOOKING and SERVICE, is resolved through SERVICE_REQUEST. The diagram therefore maps directly onto the nine tables and nine foreign keys described in Section 3.

SECTION 5

Workflow

This section describes how the Hotel Management System is used during a guest's stay, from the first enquiry to check-out and reporting. Each stage reads from or writes to specific tables of the database. The flowchart in Figure 3 shows the sequence of stages and the two decision points: whether a suitable room is available, and whether payment is complete.

5.1 Overview of the Stages

The table below summarises who performs each stage, the tables it uses and the kind of SQL operation involved. Each stage is explained in detail in Section 5.3.

Stage	Performed by	Main tables	SQL operation
1. Register guest	Front-office staff	GUEST	INSERT (or SELECT if already registered)
2. Check availability and room type	Front-office staff	ROOM, ROOM_TYPE, BOOKING	SELECT with JOIN
3. Create booking	Front-office staff	BOOKING	INSERT
4. Record check-in / check-out dates	Front-office staff	BOOKING, ROOM	INSERT / UPDATE
5. Handle service requests	Service staff	SERVICE_REQUEST, SERVICE, EMPLOYEE	INSERT
6. Calculate charges	Accounts	BOOKING, ROOM_TYPE, SERVICE_REQUEST, SERVICE	SELECT with JOIN and SUM
7. Record payments	Accounts	PAYMENT	INSERT; SELECT SUM to check balance
8. Update room status and reports	Front office and management	BOOKING, ROOM and others	UPDATE; SELECT with GROUP BY, HAVING, sub-queries

5.2 Workflow Flowchart

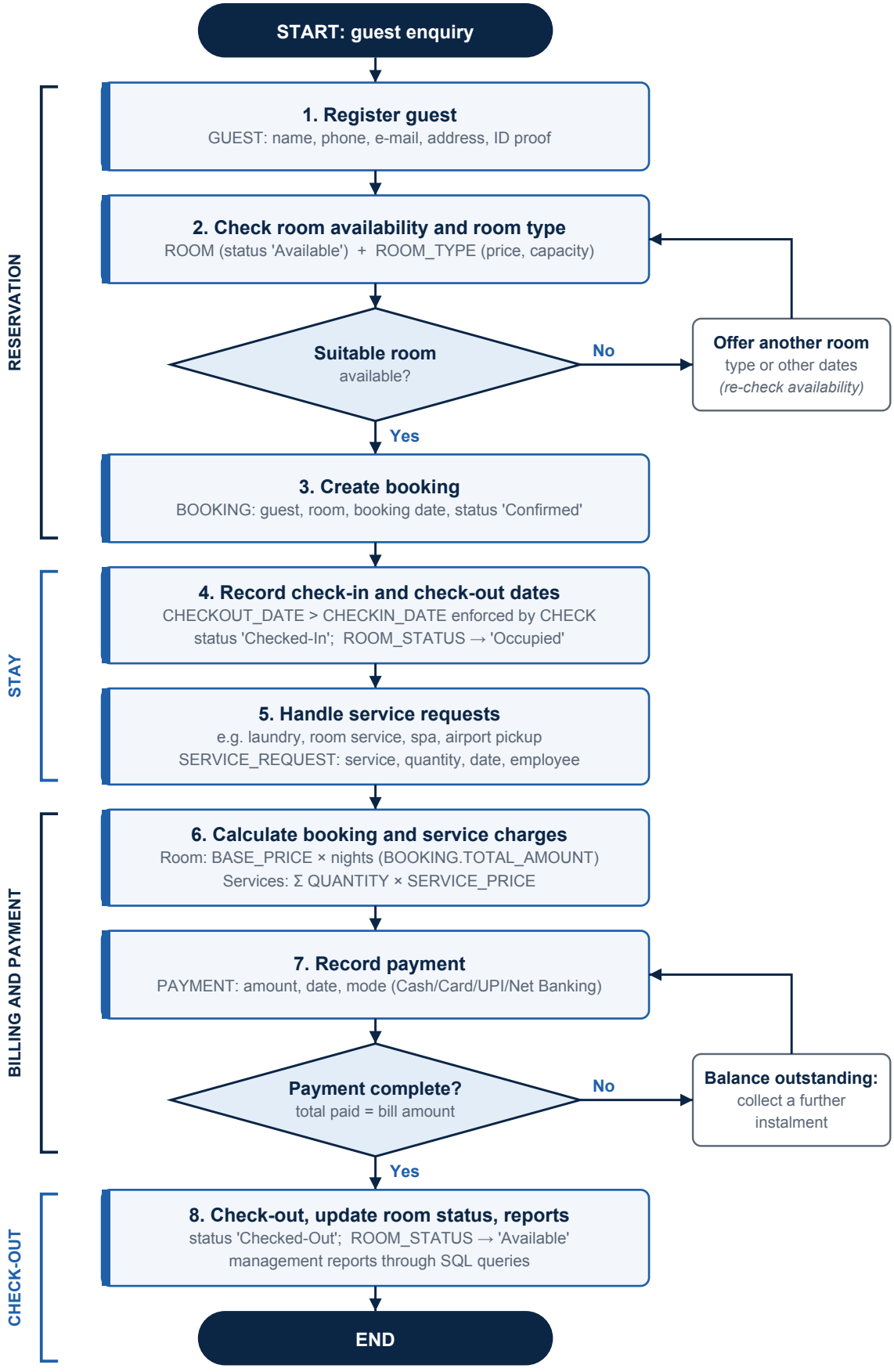


Figure 3: Workflow of the Hotel Management System

5.3 Explanation of Each Stage

Stage 1 — Register a guest

When a guest makes an enquiry, the front-office staff first check whether the guest is already recorded. A new guest is inserted into GUEST with name, phone number, e-mail, address, city and identity-proof number. The UNIQUE constraints on PHONE, EMAIL and ID_PROOF_NO prevent the same person from being registered twice.

Tables involved: GUEST

Stage 2 — Check room availability and room type

The staff query ROOM joined with ROOM_TYPE to find rooms whose status is ‘Available’ and whose category suits the guest’s needs in terms of tariff (BASE_PRICE) and occupancy (CAPACITY). Rooms under ‘Maintenance’ are excluded, and existing bookings for the requested dates are consulted. **Decision:** if no suitable room is available, the guest is offered another room type or other dates and availability is checked again; if a room is available, the process continues.

Tables involved: ROOM, ROOM_TYPE, BOOKING

Stage 3 — Create a booking

A row is inserted into BOOKING linking the guest (GUEST_ID) to the selected room (ROOM_ID), with the booking date. BOOKING_STATUS takes its default value, ‘Confirmed’. A booking that does not proceed can later be marked ‘Cancelled’. The foreign keys guarantee that the guest and room exist.

Tables involved: BOOKING

Stage 4 — Record check-in and check-out dates

CHECKIN_DATE and CHECKOUT_DATE define the period of stay; the table-level CHECK constraint rejects any check-out date that is not later than the check-in date. When the guest arrives, BOOKING_STATUS is updated to ‘Checked-In’ and the room’s ROOM_STATUS to ‘Occupied’.

Tables involved: BOOKING, ROOM

Stage 5 — Handle service requests

During the stay, each request for a chargeable service, such as laundry, room service, spa, airport pickup or the conference hall, is recorded as a row in SERVICE_REQUEST with the booking, the service, the quantity, the request date and the employee who attended it. A booking may raise any number of requests.

Tables involved: SERVICE_REQUEST, SERVICE, EMPLOYEE

Stage 6 — Calculate booking and service charges

The room charge is the category tariff multiplied by the number of nights, BASE_PRICE × (CHECKOUT_DATE − CHECKIN_DATE), and is held in BOOKING.TOTAL_AMOUNT. In the sample data, a Deluxe room at Rs. 4,000 for two nights gives Rs. 8,000. Service charges are obtained by joining SERVICE_REQUEST with SERVICE and summing QUANTITY × SERVICE_PRICE; for the same booking, two room-service orders at Rs. 500 and three laundry orders at Rs. 300 give Rs. 1,900. The bill amount is the room charge plus the service charges.

Tables involved: BOOKING, ROOM_TYPE, SERVICE_REQUEST, SERVICE

Stage 7 — Record payments

Each payment is inserted into PAYMENT with the amount, date and mode (Cash, Card, UPI or Net Banking) against the booking. A booking may be paid in instalments; for example, sample booking 1 has an advance by UPI and a later payment by card. **Decision:** the total of PAYMENT.AMOUNT for the booking is compared with the bill amount; if a balance is outstanding, a further payment is collected and recorded, otherwise the process continues to check-out.

Tables involved: PAYMENT

Stage 8 — Update room status and retrieve management reports

At check-out, BOOKING_STATUS is set to 'Checked-Out' and the room's ROOM_STATUS returns to 'Available' so that it can be allotted again. Management then retrieves reports with SQL, such as revenue by room type, room occupancy by status, highest-spending guests, payment totals by mode, department-wise employee count and salary expenditure, and the most frequently requested services.

Tables involved: BOOKING, ROOM, and all tables for reports

5.4 Status Changes During the Workflow

The two status attributes of the schema record where each booking and room stands in the workflow:

Stage	BOOKING_STATUS	ROOM_STATUS
Booking created (Stage 3)	Confirmed (default)	unchanged until arrival
Guest checks in (Stage 4)	Checked-In	Occupied
Guest checks out (Stage 8)	Checked-Out	Available
Booking withdrawn	Cancelled	unchanged
Room under repair	—	Maintenance (excluded from availability)